

Project ACSAR-E

AUTONOMOUS COGNITIVE SITUATIONAL AWARENESS ROBOT

June 2026 Monthly Progress Report

WSO2 Industry Engagement & DASH Ground Control Station Development

Authors:
Thilac R.
Dakshayani R.
Daniel
Rashad

Affiliation:
Faculty of Engineering
University of Peradeniya
Sri Lanka

June 2026

Contents

Executive Overview	3
Introduction	3
Reporting Scope	3
Monthly Achievements Summary	3
SLAM Mapping and Environment Reconstruction	4
Introduction and Background	4
What is SLAM?	4
The ASCAR-E SLAM Mission	4
Hardware Configuration	4
System Architecture Overview	4
Phase 1: Hardware and Network Integration	5
Network Topology	5
The Raw ROS 2 Topics	5
Complete ROS 2 Topic List	5
Phase 2: ROS 2 Data Acquisition & Pre-processing (<code>lidar_bridge.py</code>)	5
Odometry Caching and Quaternions	6
Strict Outlier Rejection	6
Voxel Downsampling	6
Phase 3: The Network Transport Layer (MQTT)	6
Phase 4: Point Cloud Processing & Z-Band Filtering (<code>blender_mcp.py</code>)	7
The World Frame Advantage	7
The Z-Band Filter	7
Phase 5: The 2D Occupancy Grid Mapping Algorithm	8
Projection to 2D	8
The Occupancy Dictionary	8
Phase 6: The “Two-Test” Wall Classification Algorithm	8
Test 1: The Confidence Filter (<code>count</code> ≥ 10)	8
Test 2: The Height-Span Filter ($\Delta z \geq 0.60$ m)	9
Phase 7: BMesh 3D Geometry Extrusion in Blender	9
The Performance Bottleneck	9
The Batched BMesh Solution	10
The Aesthetic Fill Factor	10
Phase 8: Quadruped Odometry & Path Tracking	10
The Quadruped Z-Axis Problem	10
Planar Clamping and Decimation	10
Mapping Results	11
Technical Challenges	11
System Tuning for Different Environments	11
DASH Ground Control Station Development	12
Overview	12
System Architecture	12
Dynamic Preflight Environment Checks & Auto-Repair	13
Network Troubleshooter	13
Windows XInput Gamepad Interface	14
High-Fidelity Mock Engine & Telemetry Bridge	14

Onboard SBC Telemetry Daemon (<code>dash_bridge.py</code>)	14
3D OpenGL Point Cloud Viewport	15
Computer Vision & Live Camera Stream	15
Embedded FastAPI SDK Server	15
SQLite Schema & Settings Manager	15
Standalone Executable Packaging	16
Packaging Pipeline Mechanics	16
Development Progress Summary	17
Robot Integration & Connectivity	17
Sensor Data Retrieval & Verification	17
ROS2 Integration	17
SLAM Pipeline	18
Field Testing Notes	18
DASH Ground Control Station	18
Identified Problems	19
1. Unnecessary Wobblings in the Robot	19
2. Staircase Climbing Limitations Encountered During Field Deployment	19
3. Issues Identified from the Interns' Code	19
4. Issues Found in the Setup Documentation Provided	20
Repository Links	21
Conclusion	22
Progress Summary	22
Key Technical Takeaways	22
System Tuning Conclusion	22

Executive Overview

Introduction

This report summarises the research, integration, and deployment progress achieved during June 2026 by the **ACSAR-E (Autonomous Cognitive Situational Awareness Robot)** development team. The project team — Thilac R., Dakshayani R., Daniel, and Rashad — conducted two field deployments at the WSO2 Lanka (Pvt) Ltd headquarters in Colombo and simultaneously developed the DASH ground control station software.

Reporting Scope

This report covers all system integration, field deployment, mapping validation, and software engineering tasks performed during June 2026. The scope includes field visits to the WSO2 Colombo headquarters, analysis of the generated structural point clouds, and the development of the DASH ground control platform.

Monthly Achievements Summary

Core Monthly Achievements

- **Unitree Go2 EDU Field Integration:** Successfully resolved DDS network packet drop errors and clock drift anomalies over corporate Access Points during field deployment at the WSO2 facility.
- **SLAM Pipeline Implementation:** Designed and deployed a downsampled LiDAR and odometry ingestion bridge running at 10 Hz, transmitting structural data via Mosquitto MQTT to a Blender MCP receiver.
- **WSO2 Floor Mapping Validation:** Conducted two-floors office scanning sessions at the WSO2 facility, generating a stylised 3D voxel reconstruction while filtering dynamic obstacles and floor noise.
- **DASH Ground Control Station:** Implemented a multi-tab multi-threaded PyQt5 interface featuring a GPU-accelerated OpenGL point cloud viewer, automated Windows Firewall repair tools, and an embedded FastAPI SDK, packaged as a standalone `DASH_Control_Center.exe`.

SLAM Mapping and Environment Reconstruction

Introduction and Background

What is SLAM?

Simultaneous Localisation and Mapping (SLAM) is the computational problem of constructing or updating a map of an unknown environment while simultaneously keeping track of the agent's location within it. In the context of the ASCAR-E project, the agent is a quadrupedal robot (Unitree Go2) exploring an indoor corporate environment.

The ASCAR-E SLAM Mission

The primary objective of the ASCAR-E SLAM pipeline is to take dense, noisy, and high-bandwidth 3D LiDAR point clouds and odometry data from a roaming robot, compress and filter it in real-time, and transmit it to a remote workstation running Blender. Inside Blender, the system programmatically generates a clean, stylised, structural 3D model of the room — walls and pillars — and the robot's traversed path, discarding transient noise such as walking humans, sensor ghosting, and floor reflections.

Hardware Configuration

- **Robot Platform:** Unitree Go2.
- **Sensors:**
 1. **Hesai XT16 3D LiDAR (Primary):** The primary 360° long-range scanning sensor used for global room reconstruction.
 2. **Unitree 4D LiDAR L1 (Built-in):** An ultra-wide hemispherical LiDAR primarily used for near-field obstacle avoidance and local terrain mapping.
 3. **Front Depth / RGB Camera:** Provides visual feed for object detection and visual-inertial odometry (VIO).
 4. **Integrated IMU:** A high-frequency (250 Hz+) inertial measurement unit used for hardware-level point cloud deskewing and state estimation.
- **Compute:** Internal ARM-based compute board running ROS 2.

System Architecture Overview

The ASCAR-E SLAM pipeline is divided into two computational domains to prevent bottlenecking the robot's internal CPU and to avoid overwhelming the wireless network.

1. **The ROS 2 Publisher Node (`lidar_bridge.py`):** Runs directly on the robot. It hooks into the raw ROS 2 topics, performs heavy downsampling and mathematical outlier rejection, and bridges the data out of the ROS ecosystem via MQTT.
2. **The Message Broker (Mosquitto MQTT):** Acts as the network transport middleman. By using MQTT instead of native ROS 2 DDS over Wi-Fi, the system avoids UDP multicast packet loss and ensures lightweight, reliable delivery of JSON payloads to the topic `ascar/lidar`.

3. **The Blender MCP Mapper (`blender_mcp.py`):** Runs on the remote workstation inside the Blender 3D modelling software’s internal Python environment, applying spatial classification algorithms to build the final mesh.

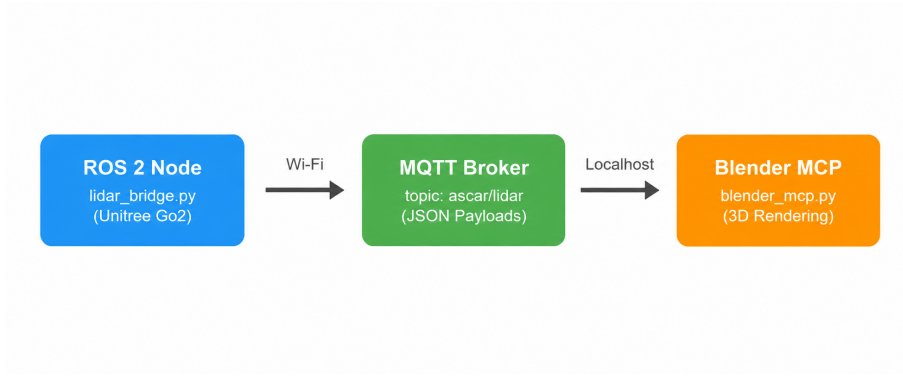


Figure 1: ASCAR-E SLAM System Architecture

Phase 1: Hardware and Network Integration

Before algorithms can run, physical and network connections must be established. The Unitree Go2 natively broadcasts its data over a localised Wi-Fi hotspot.

Network Topology

The remote workstation connects to the robot’s **Unitree_Go2** network. The ROS 2 environment on the PC is configured to match the robot’s Domain ID (typically `ROS_DOMAIN_ID=0`). This allows the PC to natively observe the `sensor_msgs/msg/PointCloud2` and `nav_msgs/msg/Odometry` topics without manual socket programming.

The Raw ROS 2 Topics

- `rt/utlidar/cloud_deskewed` (alias `/utlidar/cloud_deskewed`): The raw point cloud. Because the robot is moving while the LiDAR spins, the points captured at the start of a spin are spatially offset from those at the end. The Go2’s internal firmware uses IMU data to deskew (correct) the cloud before publishing it.
- `rt/utlidar/robot_odom`: The Visual-Inertial Odometry (VIO) combined with leg kinematics, providing the robot’s world-frame pose.

Complete ROS 2 Topic List

The complete list of ROS 2 topics is documented in the following repository:

<https://github.com/this-isnt-daniel/unitree-ros2-bridge>

Phase 2: ROS 2 Data Acquisition & Pre-processing (`lidar_bridge.py`)

The raw LiDAR generates upwards of 30,000 points per frame at 10 frames per second. The `lidar_bridge.py` script solves the resulting bandwidth problem through downsampling and compression.

Odometry Caching and Quaternions

The bridge subscribes to `/utlidar/robot_odom` and caches the full 3D translation (`px`, `py`, `pz`) and the full Quaternion orientation (`qx`, `qy`, `qz`, `qw`). Unlike a wheeled rover that stays flat on the floor, a walking quadruped pitches and rolls with every step, making full quaternion caching essential. This cached pose is injected into every single LiDAR frame payload.

```
def odom_callback(self, msg: Odometry):
    pos = msg.pose.pose.position
    ori = msg.pose.pose.orientation
    self._pose = {
        "px": pos.x, "py": pos.y, "pz": pos.z,
        "qx": ori.x, "qy": ori.y, "qz": ori.z, "qw": ori.w
    }
```

Strict Outlier Rejection

LiDAR sensors can output NaN or Inf values when a laser pulse does not return, and multi-path reflections create ghost points far outside the operational zone. The code applies a strict mathematical mask:

```
finite_mask = np.all(np.isfinite(pts), axis=1)
range_mask = np.linalg.norm(pts, axis=1) < self.max_range # Default
15.0m
pts = pts[finite_mask & range_mask]
```

Voxel Downsampling

The bridge uses the Open3D library to perform voxel downsampling. Points that fall into the same 3D voxel cell are averaged into a single representative point at the centroid, reducing the point cloud density by 80–95% while preserving structural outlines. When Open3D is unavailable, a NumPy grid-uniqueness fallback is used.

```
if OPEN3D_AVAILABLE:
    cloud = o3d.geometry.PointCloud()
    cloud.points = o3d.utility.Vector3dVector(pts)
    try:
        ds = cloud.voxel_down_sample(voxel_size=self.voxel_size)
        pts_out = np.asarray(ds.points).tolist()
    except RuntimeError as e:
        self.get_logger().warn(f"Voxel downsample failed: {e} - using raw")
        pts_out = pts.tolist()
else:
    grid = np.floor(pts / self.voxel_size).astype(int)
    _, idx = np.unique(grid, axis=0, return_index=True)
    pts_out = pts[idx].tolist()
```

Figure 2: Open3D Voxel Downsampling Implementation Screenshot

Phase 3: The Network Transport Layer (MQTT)

`lidar_bridge.py` converts the ROS 2 `PointCloud2` binary buffers into a JSON string and publishes it to the Mosquitto MQTT broker on the topic `ascar/lidar`. MQTT is used rather than native ROS 2 DDS because Blender's isolated Python environment cannot easily run ROS 2 RCLPY libraries natively, especially on Windows.

The full JSON telemetry payload structure published on `ascar/lidar` is as follows:

```
rashad@ubuntu:~$ mosquitto_sub -h 127.0.0.1 -t ascar/lidar
{
  "stamp_sec": 1718005432,
  "stamp_nsec": 450000000,
  "frame_id": "odom",
  "n_raw": 15420,
  "pose": {
    "px": 1.245,
    "py": 0.521,
    "pz": 0.051,
    "qx": 0.0,
    "qy": 0.0,
    "qz": 0.342,
    "qw": 0.939
  },
  "has_odom": true,
  "points": [
    [
      1.04,
      2.01,
      0.55
```

Figure 3: Mosquitto MQTT Telemetry Subscription Terminal Screenshot

Phase 4: Point Cloud Processing & Z-Band Filtering (blender_mcp.py)

Inside Blender, a background thread listens to MQTT and queues the incoming JSON frames. When the Blender UI timer fires at 10 Hz, it processes the queued points.

The World Frame Advantage

The `/utlidar/cloud_deskewed` topic is already transformed into the `odom` (world) frame by the robot's internal hardware. The points received in Blender do *not* need to be multiplied by the robot's rotation matrix; `blender_mcp.py` explicitly skips double-transforming the points.

The Z-Band Filter

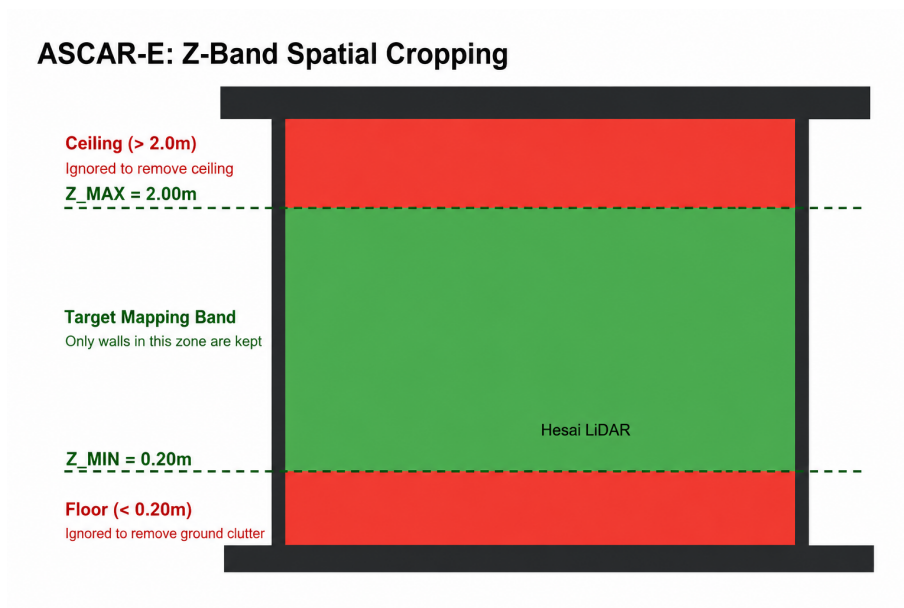


Figure 4: ASCAR-E Z-Band Spatial Cropping Layout

To prevent rendering the flat floor and the ceiling, the script isolates a horizontal band of space between 20 cm (safely above the floor) and 2.00 m (below the ceiling):

```
WORLD_Z_MIN = 0.20 # metres - safely above the floor
WORLD_Z_MAX = 2.00 # metres - below the ceiling
z_mask = (world_pts[:, 2] > WORLD_Z_MIN) & (world_pts[:, 2] <
WORLD_Z_MAX)
```

Any point lower than 20 cm or higher than 2.00 m is discarded from memory.

Phase 5: The 2D Occupancy Grid Mapping Algorithm

Projection to 2D

Every point that survives the Z-Band filter is projected onto the XY plane and quantised into a discrete grid of 15 cm $\times$ 15 cm cells ($\text{CELL_SIZE} = 0.15$):

```
ix_arr = np.floor(world_pts[:, 0] / CELL_SIZE).astype(np.int32)
iy_arr = np.floor(world_pts[:, 1] / CELL_SIZE).astype(np.int32)
```

The Occupancy Dictionary

The script maintains a global Python dictionary `_occupancy_2d`. The key is the cell coordinate (`ix`, `iy`). The value is a list `[confidence_count, z_min_seen, z_max_seen]`. Every time a point lands in a cell, the system increments `confidence_count` and updates `z_min` and `z_max` if the new point extends the cell's vertical bounds.

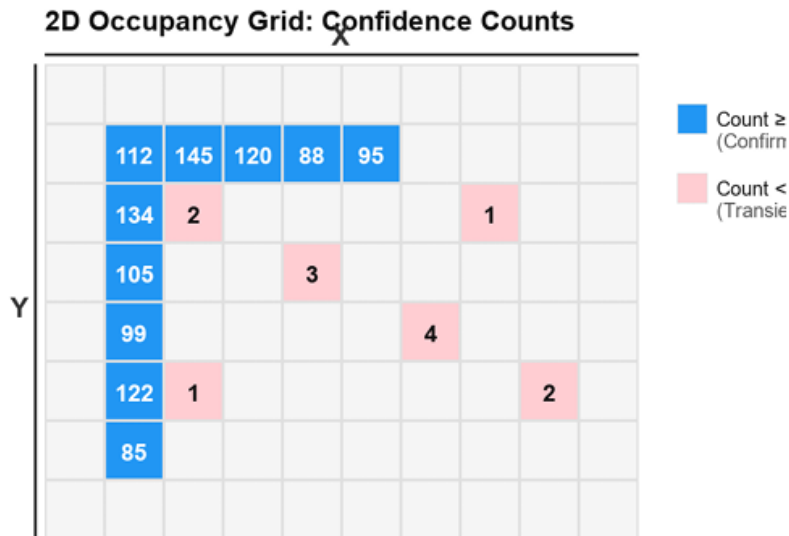


Figure 5: 2D Occupancy Grid Confidence Counts Mapping Matrix

Phase 6: The “Two-Test” Wall Classification Algorithm

This is the intellectual core of the ASCAR-E mapping system. Before a cell in the 2D occupancy grid is drawn as a 3D wall in Blender, it must pass two simultaneous tests.

Test 1: The Confidence Filter (count ≥ 10)

A cell must have been hit by a laser at least `MIN_CELL_CONFIDENCE` times.

- **The Problem:** As the robot walks, it sees a human walk past. The human generates LiDAR points. If those points were drawn, the map would have ghost trails of people.
- **The Solution:** Because the human is moving, they only occupy a specific 15 cm cell for a fraction of a second, generating only 3–4 laser hits. A stationary wall accumulates hundreds of hits over time. Requiring at least 10 hits filters out dynamic moving objects while preserving structural elements.

Test 2: The Height-Span Filter ($\Delta z \geq 0.60$ m)

- **The Problem:** Sensor noise causes floor reflections. Even with the Z-Band filter, a bad reflection might register at $Z = 0.25$ m. Because the floor is stationary, this reflection accumulates hundreds of hits, passing Test 1.
- **The Solution:** A floor reflection is flat — it will have a `z_min` of 0.25 m and a `z_max` of maybe 0.28 m, giving a vertical span of only 3 cm. A structural wall extends vertically. By enforcing:

$$\Delta z = z_{\max_seen} - z_{\min_seen} \geq 0.60 \text{ m} \quad (1)$$

the system discards floor noise, table legs, cable trays, and low-lying clutter, preserving only structural vertical elements.

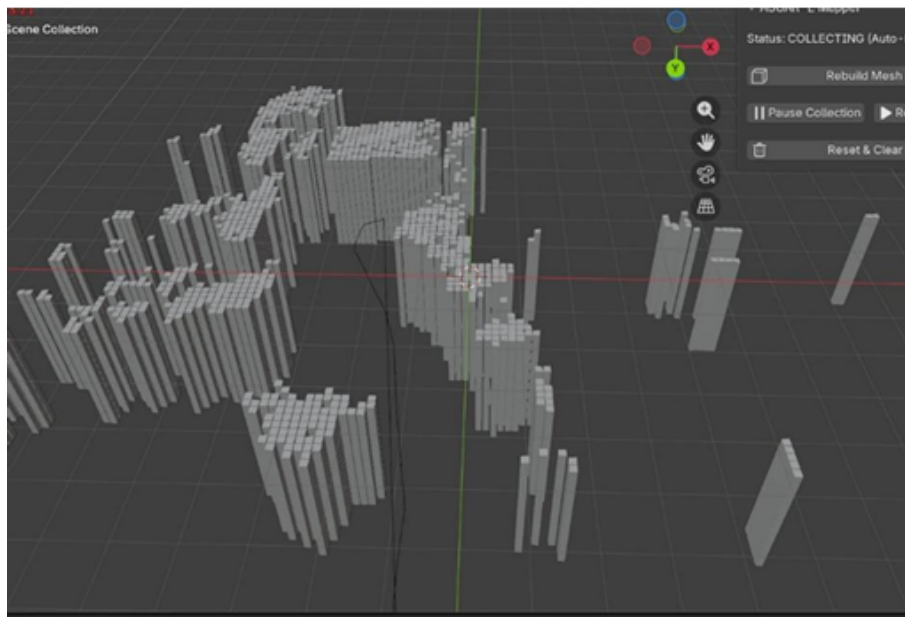


Figure 6: Real-Time Wall Classification and Filtration Output in Blender

Phase 7: BMesh 3D Geometry Extrusion in Blender

The Performance Bottleneck

If Blender’s API is called via `bpy.ops.mesh.primitive_cube_add()` once per cell for a large room, the UI will freeze for minutes and the application will crash. Every Blender object carries significant overhead in transform matrices and dependency graph nodes.

The Batched BMesh Solution

Rather than adding cubes individually, the ASCAR-E script uses `bmesh` — Blender’s internal C-level mesh API — to construct the entire room as a single unified mesh object. For every qualifying cell, the script calculates its real-world centre (`cx`, `cy`) and creates 8 vertices forming a rectangular column extruded to a wall height of `WALL_EXTRUDE_HEIGHT = 2.60m`:

```
h = WALL_EXTRUDE_HEIGHT # 2.60m
half = (CELL_SIZE * WALL_COLUMN_FILL) / 2.0
# Bottom Vertices
v0 = bm.verts.new((cx - half, cy - half, 0.0))
v1 = bm.verts.new((cx + half, cy - half, 0.0))
v2 = bm.verts.new((cx + half, cy + half, 0.0))
v3 = bm.verts.new((cx - half, cy + half, 0.0))
# Top Vertices
v4 = bm.verts.new((cx - half, cy - half, h))
v5 = bm.verts.new((cx + half, cy - half, h))
v6 = bm.verts.new((cx + half, cy + half, h))
v7 = bm.verts.new((cx - half, cy + half, h))
```

The Aesthetic Fill Factor

`WALL_COLUMN_FILL = 0.90` causes each column to be 13.5 cm wide rather than filling the full 15 cm cell, leaving a 10% gap between adjacent columns. This produces a stylised, voxel-like rendering of the map.

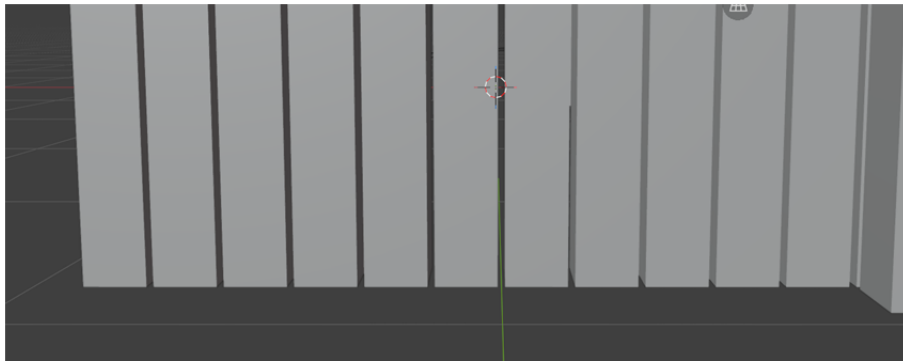


Figure 7: Voxel-like Column Gap: Aesthetic Fill Factor (`WALL_COLUMN_FILL = 0.90`)

Phase 8: Quadruped Odometry & Path Tracking

The Quadruped Z-Axis Problem

A quadruped robot bounces vertically with every step. Plotting raw 3D odometry produces a jagged sawtooth wave that is visually distracting and inaccurate as a 2D floor path.

Planar Clamping and Decimation

The path tracking function solves this through two mechanisms:

1. **Decimation:** A point is only appended if the robot has moved more than `PATH_MIN_MOVE_M` (8 cm) from the last recorded point, calculated by Euclidean distance.
2. **Z-Clamping:** The odometry `pz` value is entirely ignored. All path vertices are forced to a flat plane at `PATH_Z_FIXED` (5 cm above the floor).

```
# Decimation check: only append if Euclidean distance exceeds 8cm
distance = np.sqrt((px - last_x)**2 + (py - last_y)**2)
if distance > 0.08:
    # Z-clamping: force coordinates to flat plane 5cm above floor
    bm.verts.new((px, py, 0.05))
```

This results in a smooth, continuous path line tracing the robot's journey around the room.

Mapping Results

During the field deployments at the WSO2 facility, the robot successfully scanned multiple office zones across two floors. The resulting 3D geometry accurately represented cubicle layouts and corridors.

Technical Challenges

The primary mapping challenge stemmed from the highly reflective glass panels used as workspace dividers. Laser beams either passed through the glass or reflected at sharp angles, creating duplicate point arrays (multipath noise) and false walls. High-frequency walking gaits also caused significant sensor vibration that degraded odometry estimation.

System Tuning for Different Environments

SLAM Environment Tuning Parameters

- **Warehouse Mapping:** Increase MAX_SENSOR_RANGE to 30.0 m and CELL_SIZE to 0.30 m to map large open spaces quickly.
- **Cluttered Office:** Increase MIN_HEIGHT_SPAN to 0.80 m so that desks and office equipment are not misclassified as structural walls.
- **Crowded Environment:** Raise MIN_CELL_CONFIDENCE to 30 to aggressively filter out dense groups of moving people.

DASH Ground Control Station Development

Overview

DASH (Desktop Assistant & Robotics Shell) is a desktop ground control station (GCS) designed for the Unitree Go2 EDU quadruped. It bridges the gap between Windows desktop hosts and the robot's Linux SBC running ROS2. The application is built using a Python-PyQt5 frontend framework, leveraging standard TCP stream sockets via the Python built-in `socket` library, `ctypes` for Windows DLL interfacing, FastAPI, OpenGL via `pyqtgraph`, and computer vision via OpenCV. DASH is packaged via PyInstaller into a standalone, portable Windows executable (`DASH_Control_Center.exe`). The SQLite database is stored in the user's home directory at `~/dash_robotics/dash.db` to ensure database writes persist across sessions, along with an embedded web server and binary assets.

System Architecture

The DASH application uses a multi-threaded design to keep the GUI responsive while handling high-rate background data. The interaction between the packaged Windows executable, the local API server, and the onboard Unitree Go2 SBC is structured as follows:

- **Windows Host (DASH Standalone Executable):** `main.py` entrypoint launches a PyQt5 `MainWindow` with a 15-tab stacked page layout. Subsystem threads — Gamepad (XInput DLL), Camera (OpenCV + Haar Cascades), 3D Point Cloud Viewport (OpenGL/PyQt-Graph), Telemetry Bridge (Socket Client), and FastAPI/WebSocket Web Server — all run concurrently while sharing a local SQLite database.
- **Unitree Go2 SBC (Onboard Linux Computer):** The `dash_bridge.py` daemon receives TCP socket connections on port 5555 and publishes ROS2 topic data (`/imu/data`, `/odom`, `/utlidar/cloud`, `/battery_state`) along with hardware diagnostics from SysFS.
- **External Client Systems:** The embedded FastAPI server exposes REST and WebSocket endpoints, allowing external SDK scripts (Python/JS) to stream live telemetry at 10 Hz.
- **Camera & SSH:** The camera panel connects via RTSP (port 8554) to the robot's live video feed. SSH/SFTP control operates over port 22 to the robot's SSH server daemon.

The key subsystems are summarised below:

Subsystem	Implementation	Technical Function
User Interface	PyQt5 QStackedWidget (15 tabs)	Renders dashboard panels and health gauges.
Gamepad Thread	XInput DLL via <code>ctypes</code>	Polls controller inputs.
Camera Ingestion	OpenCV RTSP VideoCapture	Retrieves and processes video frames.
3D Viewport	PyQtGraph OpenGL Widget	Renders downsampled LiDAR point clouds on GPU.
Telemetry Bridge	TCP Socket Client	Receives JSON frames from robot SBC on port 5555.
Local Database	SQLite	Manages profiles, settings, and command logs.
Web Server	FastAPI & Uvicorn Thread	Exposes REST and WebSocket API endpoints.

Dynamic Preflight Environment Checks & Auto-Repair

The preflight checker (`app/core/preflight_check.py`) scans the local host environment for runtime compatibility, reporting statuses dynamically to the splash screen:

- **Operating System:** Verifies that the platform is Windows and prints the release details. A specific Windows 10/11 version assertion is not enforced.
- **Python Engine:** Confirms Python `>= 3.8`.
- **Hardware Allocations:** Probes CPU utilisation and available RAM using `psutil`.
- **Disk Space:** Warns if free disk space falls below 5 GB; critical alert below 2 GB.
- **WSL Interface:** Searches for the `ws1.exe` binary in system PATH and verifies the presence of Ubuntu distributions via subprocess.
- **Windows Firewall Diagnostics & Auto-Repair:** Probes firewall profiles using `netsh advfirewall`. If blocked, the user can trigger an Auto-Repair that calls elevated subprocesses to register inbound UDP/TCP rule profiles allowing ROS2 DDS traffic on ports 7400–7500.
- **Network & VPN Detection:** Evaluates system network adapters and alerts the user of any active VPN virtual interfaces (e.g. WireGuard, Tap, Tun) which block DDS multicast traffic.
- **Time Synchronisation:** Queries the Windows Time Service (`w32tm`). Accurate time sync is critical for robot TF coordinate transformation matching. If out of sync, DASH triggers an administrative repair: `w32tm /resync`.
- **USB Subsystem:** Scans the Windows Registry key `HARDWARE\DEVICEMAP\SERIALCOMM` via `winreg` to map active COM/serial interfaces.

Network Troubleshooter

The network diagnostics engine (`app/core/network_troubleshooter.py`) runs tests asynchronously:

1. **Subnet Matching:** Computes subnets by applying network masks to host IPs and compares them to the robot's configured IP (192.168.123.18). If there is a mismatch, it advises configuring a static IP.

2. **ICMP Ping:** Pings the robot, parsing output strings to compute average latency and packet loss.
3. **Port Scanner:** Performs socket connect handshakes to verify that SSH (Port 22) and the Telemetry Bridge (Port 5555) are listening.

Windows XInput Gamepad Interface

The controller manager (`app/core/gamepad.py`) interacts with physical gamepads at the kernel level:

- **Dynamic Driver Loading:** Uses `ctypes.windll` to interface directly with standard Windows OS gamepad DLL runtimes (`xinput1_4.dll`, `xinput1_3.dll`, or `xinput9_1_0.dll`). No custom C-bindings are compiled; `ctypes` loads the system-provided DLLs directly.
- **Analog Normalisation:** Polls analog sticks (16-bit signed shorts) and applies a configurable deadzone of 8000. Raw input is rescaled to linear floating-point values between -1.0 and 1.0 .
- **Asynchronous Threading:** Runs on a dedicated `QThread` polling at ~ 33 Hz (30 ms interval).

High-Fidelity Mock Engine & Telemetry Bridge

The telemetry bridge (`app/core/telemetry_bridge.py`) operates in two modes:

1. **Physical Mode:** Establishes a TCP socket connection to the robot, reading line-separated JSON payloads from port 5555.
2. **Simulation Mode:** Runs an offline telemetry simulation engine that generates:
 - **Inertial Walk:** Oscillating pitch/roll orientations calculated using pure sine and cosine mathematical offsets to simulate quadruped gait dynamics.
 - **Kinematic Joint Drive:** Simulated joint states for 12 motors (FR, FL, RR, RL legs) mapping walk-cycle trajectories.
 - **Occupancy Grid Simulator:** Simulates a 120×120 2D costmap. As the robot traverses a predefined trajectory, the engine exposes obstacles from a ground-truth layout using a search-radius sweep.
 - **3D Point Cloud Generator:** Generates 1,500 points in cylindrical wall, ceiling, and floor patterns. It applies a rotation matrix to simulate active LiDAR spin.

Onboard SBC Telemetry Daemon (`dash_bridge.py`)

This script resides on the robot SBC and acts as the data publisher:

- **ROS2 Subscriptions:** Dynamically loads `rclpy` and subscribes to:
 - `/imu/data` (`sensor_msgs/msg/Imu`)
 - `/odom` (`nav_msgs/msg/Odometry`)
 - `/battery_state` (`sensor_msgs/msg/BatteryState`)
 - `/utlidar/cloud` (`sensor_msgs/msg/PointCloud2`)

- **LiDAR Downsampling:** High-frequency point clouds are downsampled by a factor of $15\times$ using struct binary unpacking (`struct.unpack_from('fff', ...)`).
- **Hardware Diagnostics:** Polls `/proc/loadavg` for CPU utilisation, `/proc/meminfo` for memory utilisation, and `/sys/class/thermal/thermal_zone0/temp` for SoC temperature.
- **TCP Publisher:** Bundles all data into a JSON structure and broadcasts it to connected clients over TCP port 5555 at 10 Hz.

3D OpenGL Point Cloud Viewport

The LiDAR visualiser (`PointCloudWidget`) renders high-performance graphics using `pyqtgraph.opengl` to offload point cloud visualisation to the host GPU. The downsampling stride logic (`points[:4]`) and PLY file export writer are implemented in `tab_pointcloud.py`, which manages the viewport and handles data preparation before passing to `PointCloudWidget` for rendering.

Computer Vision & Live Camera Stream

The camera panel (`app/ui/tabs/tab_camera.py`) uses OpenCV's `VideoCapture` to connect to local USB devices or the robot's RTSP feed (e.g. `rtsp://192.168.123.161:8554/front`). It employs Haar Cascade Classifiers (`haarcascade_frontalface_default.xml`) to process frames in real-time, drawing green bounding boxes around detected faces. A hardcoded mock label ("HUMAN [94%]") is displayed, as standard Haar Cascades do not produce direct confidence probability outputs. Snapshots are saved as PNG images; video streams are recorded using the XVID encoder into AVI files.

Embedded FastAPI SDK Server

DASH contains a headless server (`app/core/web_api.py`) that exposes robot data for external scripting:

- **FastAPI Server:** Runs via `uvicorn` on a separate thread, auto-detecting open ports starting at 8000.
- **REST Endpoints:** Exposes live telemetry snapshots at `/api/telemetry`, `/api/system`, `/api/battery`, `/api/imu`, `/api/gps`, `/api/motors`, and `/api/motor/{id}`.
- **WebSocket Streams:** Broadcasts live 10 Hz JSON telemetry streams to all clients connected to `/ws`.
- **Automatic SDK Documentation:** Generates static HTML documentation explaining how to fetch telemetry using standard scripts.

SQLite Schema & Settings Manager

Local configurations and logs are managed via `app/core/database.py` which interfaces with SQLite. The schema is as follows:

Table	Columns	Description
settings	key (TEXT, PK), value (TEXT)	Global client setting key-value storage.
command_history	id (INT, PK), timestamp (DATETIME), command (TEXT)	CLI operation log, capped at 500 records.
saved_commands	id (INT, PK), name (TEXT, Unique), command (TEXT), description (TEXT)	Shortcut list for common ROS command sequences.
connection_profiles	id (INT, PK), profile_name (TEXT, Unique), ip_address (TEXT), ssh_port (INT), username (TEXT), password (TEXT), ros_domain_id (INT), is_active (INT)	Connection profiles for multiple robots, including SSH credentials and ROS Domain ID configuration.

Standalone Executable Packaging

To create a standalone, double-clickable application, DASH is packaged using PyInstaller:

```
pyinstaller --noconsole --onefile --name="DASH_Control_Center"
--add-data "app;app" main.py
```

Packaging Pipeline Mechanics

1. **Analysis:** Scans `main.py` and maps imports recursively. PyQt5, OpenCV, and Paramiko rely on shared C-runtimes (`.dll` files), which PyInstaller copies automatically.
2. **Resource Bundling** (`--add-data "app;app"`): Packages the entire `app` subdirectory, including Python templates, SQLite logic, UI styles, UI tabs, and assets such as `splash.png` and Haar Cascades.
3. **Single Executable Extraction** (`--onefile`): At runtime, the PyInstaller bootloader extracts all packed dependencies into a temporary directory. The project source resolves asset paths using standard relative paths based on `__file__` rather than directly referencing `sys._MEIPASS`:

$$\text{Runtime Path} = \begin{cases} \text{PyInstaller temp dir} & \text{if running compiled} \\ \text{current file directory} & \text{if running from source} \end{cases} \quad (2)$$

4. **Console Suppression** (`--noconsole`): Hides the Windows Command Prompt, executing as a standard desktop application.

Development Progress Summary

June 5, 2026

Robot Integration & Connectivity

Status: Completed

- Configured Linux development environment
- Connected Unitree Go2 robot
- Verified Ethernet communication
- Performed ping testing
- Explored Unitree mobile application
- Verified workstation connectivity

Sensor Data Retrieval & Verification

Status: Completed

- Retrieved LiDAR point cloud data
- Retrieved IMU data
- Retrieved depth camera data
- Verified sensor streams
- Initial visualisation completed

ROS2 Integration

Status: Completed

- Sensor abstraction
- Topic management
- Package organisation

June 12, 2026

SLAM Pipeline

Status: In Progress

Implemented:

- SLAM startup workflow
- Sensor integration
- Map generation

Remaining:

- Localisation testing
- Visualisation improvements

Field Testing Notes

During field testing, the robot was unable to successfully climb stairs. The failure was primarily observed at step climbing where vertical elevation exceeded the robot's effective climbing capability.

DASH Ground Control Station

Status: Completed

- PyQt5-based multi-tab desktop GUI (core structure implemented)
- Robot connection system (SSH + auto-discovery + authentication)
- Real-time telemetry streaming via TCP socket (port 5555)
- Sensor data pipeline (IMU, battery, system status)
- Logging system for events and telemetry
- Preflight environment checks (network, dependencies, ROS readiness)
- File transfer system (SFTP explorer for robot filesystem)

Status: In Progress

- Camera feed streaming with OpenCV integration
- Basic robot control interface (keyboard / gamepad support)
- LiDAR point cloud visualisation (GPU-accelerated rendering)
- Camera + LiDAR data fusion (initial projection alignment)
- Motor-level diagnostics UI (real-time torque/temp graphs)
- SLAM visualisation tab improvements (stable frame rendering)
- Sensor synchronisation system (timestamp alignment across streams)
- UI performance optimisation for high-frequency telemetry streams

Identified Problems

1. Unnecessary Wobblings in the Robot

During field deployments at the WSO2 facility on 12th June, intermittent and unexpected body oscillations were observed during certain locomotion sequences. These oscillations were not continuous; they appeared suddenly during operation and subsided after a short duration.

2. Staircase Climbing Limitations Encountered During Field Deployment

The Unitree Go2 was unable to successfully traverse the staircases encountered during field deployments at the WSO2 facility. Across all observed attempts, the robot failed to initiate a stable climbing sequence. The leading legs were unable to establish a foothold on the first step, as the foot placement did not reach or settle onto the tread surface. As a result, no valid support phase on the staircase was achieved, and the robot consistently stalled at the base and reverted to a stable standing posture.

3. Issues Identified from the Interns' Code

Several limitations were identified in the autonomy stack inherited from the previous internship team. Most of these issues originate from the incomplete migration of the CHAMP quadruped framework from ROS1 to ROS2.

- **Incomplete ROS2 Migration:** Several components of the CHAMP framework, including the velocity smoother, setup assistant, robot configuration packages, and launch files, have not yet been ported to ROS2.
- **Lack of Hardware Validation:** The autonomy stack has only been verified in Gazebo simulation. Critical components such as the hardware interface layer, sensor integration, state estimation, and locomotion performance have not been validated on the physical Unitree Go2 platform.
- **Robot Description Compatibility Issues:** Existing robot description files (URDFs) require modifications to support `ros2_control` interfaces. Consequently, additional work is required before the framework can reliably interface with real hardware.
- **Missing Robot Configuration Packages:** Configuration files containing gait parameters, controller settings, and robot-specific launch configurations are incomplete or unavailable in the ROS2 implementation.

- **Gazebo Simulation Performance Degradation:** The foot contact sensor plugin significantly reduces simulation performance, causing approximately a twofold decrease in the Real-Time Factor (RTF). As a result, contact sensing is currently disabled in simulation.
- **Codebase Refactoring Pending:** The ROS2 codebase still requires further restructuring and clean-up to conform to ROS2 development best practices and improve maintainability.

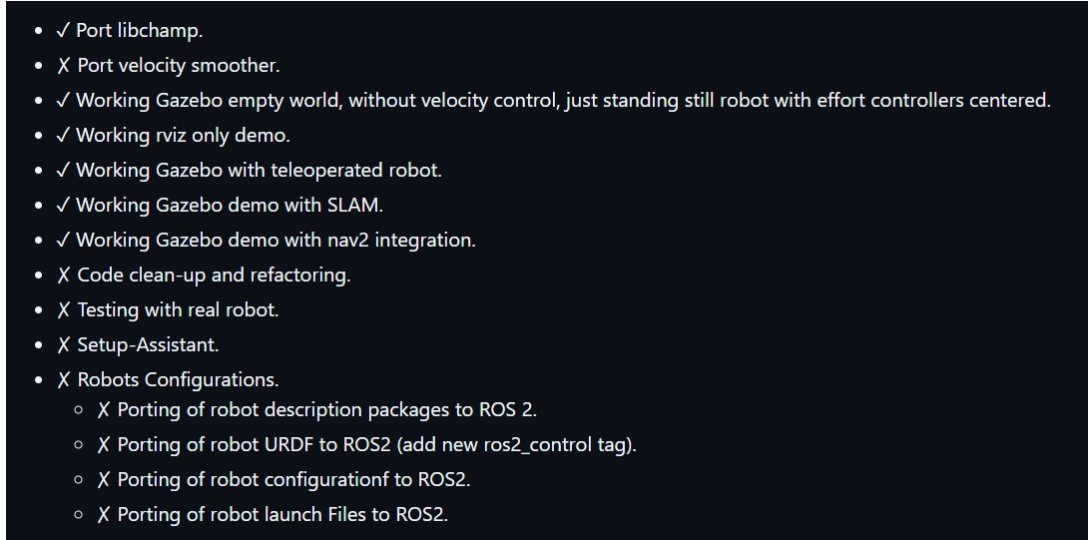


Figure 8: Current status of the CHAMP ROS2 migration as reported in the project repository. The figure highlights completed features, pending components, and functionalities that have not yet been ported or validated on real hardware.

4. Issues Found in the Setup Documentation Provided

Upon reviewing the provided setup documentation, several limitations and missing details were identified that may affect new users attempting to work with the Unitree Go2 EDU platform.

- **Insufficient Details on WebSocket-Based Communication:**
The documentation mentions the possibility of using a WebSocket-based bridge for ROS 2 topic access over Wi-Fi. However, it does not specify the implementation used, configuration steps, startup procedure, or the capabilities and limitations of the bridge. This may make it difficult for users to effectively utilize this communication method.
- **Absence of Robot Startup and Shutdown Procedures:**
Although methods for accessing the onboard Jetson are described, the documentation does not explain how to safely power on the robot, enable or disable the motors, make the robot stand or sit, or properly shut down the system. These procedures are essential for first-time users.
- **Incomplete Safety Guidelines:**
While several basic safety precautions are included, important operational safety aspects such as emergency stop procedures, battery handling precautions, and safe handling practices when the robot is powered are not discussed. Given the nature of a quadruped robot, these details are important for safe operation.
- **Lack of Sensor Integration Information:**
Additional sensors such as the Intel RealSense D435i camera and Hesai XT16 LiDAR are

listed in the documentation. However, there is no explanation regarding their physical mounting, power requirements, communication interfaces, or integration procedures with the robot.

- **Potential Restriction Due to Ubuntu 20.04 Recommendation:**

The documentation recommends Ubuntu 20.04 and ROS 2 Foxy based on observed stability. However, it does not clarify that newer operating systems such as Ubuntu 22.04 and newer ROS 2 distributions can still be used on external development machines. This may lead users to incorrectly assume that Ubuntu 20.04 is mandatory for all development environments, whereas retaining the factory Ubuntu 20.04 image is primarily important for the onboard Jetson platform.

Repository Links

The source code and project management artefacts for the ACSAR-E project are hosted on GitHub and are accessible via the following repositories:

- **DASH Ground Control Station Repository:**

<https://github.com/DakshayaniRamanesh/DASH-ASCAR-E>

- **ASCAR-E Project Board and Development Tracking:**

<https://github.com/users/DakshayaniRamanesh/projects/3>

Conclusion

Progress Summary

The developmental efforts in June 2026 represent a significant step forward for the ACSAR-E programme. Resolving initial DDS connectivity issues and completing the mapping scans across two office floors at the WSO2 facility validated the sensor integration and data processing pipeline. The development of the DASH ground control station provides a unified platform to simplify robot deployment and monitoring, with integrated troubleshooting and automated repair utilities covering firewall configuration, time synchronisation, and network diagnostics.

Key Technical Takeaways

Field testing at the WSO2 facility highlighted the differences between laboratory settings and active corporate environments. Debugging DDS multicast blocks, managing LiDAR reflections on glass partitions, and resolving clock drift synchronisation issues provided insights that directly refined the data downsampling parameters, occupancy grid algorithms, and the DASH preflight check logic.

System Tuning Conclusion

The ASCAR-E pipeline demonstrates how advanced preprocessing via Open3D voxel downsampling, combined with the logical spatial filtering of the Two-Test Wall Classification Model, can transform highly noisy, chaotic real-world sensor data into clean, architectural 3D geometry in real-time. By understanding the mathematics behind the tuning parameters, developers can adapt the system to perform effectively in any robotic deployment scenario, from open warehouses to densely occupied office environments.